

Objektovo orientované programovanie

Učiteľ:
Ing. Jozef Wagner, PhD.

Učebnica:
<https://oop.wagjo.com/>

Základné pokyny:

- Dodržiavať školský poriadok
- Pripravený na hodinu, zošit
- Ústne a písomné skúšania
- Ospravedlnenia pred začiatkom hodiny

Legenda:

- Žltý text – kód
- Červený text – poznámka do zošita
- Modré pozadie – poznámka do zošita

OPG

Teória 1

1. AI v programovaní
2. Objektovo orientované programovanie
3. Jazyk Java

AI

AI Asistent - odpovedá na otázky

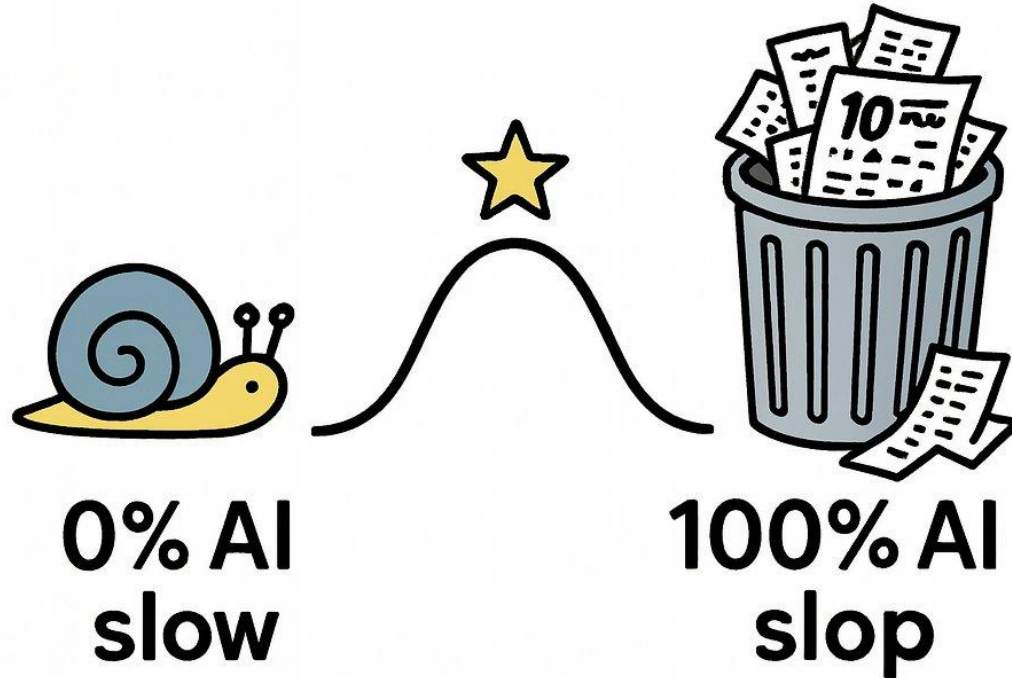
AI Agent - samostatne vykonáva úlohy

AI Slop - odpad, vznikne ak AI slepo dôverujem

AI transformovala programovanie

Vývojári trávia menej času písaním a viac recenziou a overovaním AI výstupu

The Laffer Curve, but for AI



AI Slop - odpad, vznikne ak AI slepo dôverujem

FileEditSelectionViewGoRun

←→

Search

App.javarun.shpom.xml

home > wagjo > Documents > java-fx-1 > java-fx-1 > src > main > java > com > example > App.java

1package com.example;

2

3import javafx.application.Application;

4import javafx.scene.Scene;

5import javafx.scene.control.Button;

6import javafx.scene.control.Label;

7import javafx.scene.layout.VBox;

8import javafx.stage.Stage;

9

10public class App extends Application {

11 @Override

12 public void start(Stage primaryStage) {

13 Label label = new Label("Initial message");

14 Button button = new Button("Click me");

15

16 // Placeholder for event handler, to be implemented in the next

17 button.setOnAction(e -> label.setText("Button clicked!"));

18

19 VBox root = new VBox(label, button);

20 Scene scene = new Scene(root, 300, 200);

21

22 primaryStage.setTitle("JavaFX Example");

23 primaryStage.setScene(scene);

24 primaryStage.show();

25 }

26

27 public static void main(String[] args) {

28 launch(args);

29 }

30}

31

KILO CODE

Task: Create java project that showcases basic Java FX Feature... X

46.3k131.1k\$0.27

Verify that the project runs correctly (this step may involve t... 6/7)

The apply_diff for pom.xml was successful, and I've addressed the Java version issue. The todo list needs to be updated to reflect that step 3 (adding JavaFX dependencies and configurations) is now completed. Steps 4 and 5 were completed when I modified App.java earlier, and step 6 was completed with the run script creation. Step 7 (verification) is still pending and will be attempted next. After updating the todo list, I will proceed to run the script again to verify the project.

Todo List Updated

Create a new directory named "java-fx-1" in the current workspace

Initialize a Maven project within the "java-fx-1" directory

Add JavaFX dependencies to the Maven pom.xml file

Write the main Java class that creates a simple window with a button and a label

Implement a basic event handler for the button to change the label's text when clicked

Create a shell script or batch file to run the Maven build and execute the JavaFX application

Verify that the project runs correctly (this step may involve testing in another mode)

API Request\$0.0160

Start New Task

Type a message...
(@ to add context, / for commands, hold shift to drag in files)

Code

Grok 3 Mini

Ln 1, Col 1Spaces: 4UTF-8LF{}Java

Nahradia nás AI agenti?

Programátorské miesta sa posunuli od písania kódu k **zadávaniu požiadaviek, návrhu systému a overovaniu** toho, čo vygeneruje AI.

Tradičné juniorské pozície sú ohrozené a dopyt sa sústreďuje na seniorov. Znamená to, že **seniorské zručnosti si musí budovať skôr a väčšinou sám.**

Bez znalosti základov sa robia 3 typické chyby:

- prijme sa kód, ktorý „vyzerá dobre“, ale je logicky chybný
- nevie sa správne zadať úloha. AI vyprodukuje náhodný prototyp, nezapadajúci do celkovej architektúry.
- neschopnosť opraviť zlyhanie u vecí, ktoré AI „skoro trafí“

Objektovo orientované programovanie

Paradigmy v programovaní:

- *Štruktúrované*: sekvencie príkazov, podmienky, cykly
- *Procedurálne*: podprogramy, ktoré sa vedia navzájom volať
- *Objektovo orientované*: objekty, zapuzdrenie, polymorfizmus
- *Funkcionálne*: anonymné funkcie, high-order funkcie

Objektovo orientované programovanie

Vlastnosti:

- intuitívne modely podľa nášho sveta
- zvláda väčšiu komplexnosť
- znovupoužitie

Jazyk Java

Vlastnosti:

- objektovo orientovaný
- platformovo nezávislý
- staticky typovaný: typy premenných sú kontrolované pri kompilácii

Jazyk Java

Použitie:

- webový back-end
- Android aplikácie (*jazyk Kotlin*)
- distribuované systémy (cloud)
- veľké podnikové systémy

Jazyk Java

Platformová nezávislosť:

- kód sa kompiluje do bytecode a beží nad JVM
- motto Javy: Write Once, Run Anywhere (*WORA*)
- virtuálny stroj **JVM** (*Java Virtual Machine*)